Week 1 Research

Contents

01 SECTION A
What Happens When You Type google.com? 2

02 SECTION B
ARIA Roles and Web Accessibility 3

03 SECTION C
Responsive Web Design 4

04 **Conclusion** 5

05 **References** 5

Section A: What Happens When You Type google.com?

A short walkthrough of how a browser gets a page from a server. The matching UML diagram is in browser-request-flow.pdf.

1. **URL.** `https://www.google.com:443/search?q=html#top` has a scheme (https), host (www.google.com), port (443, usually hidden), path (/search), query (?q=html) and fragment (#top). The fragment is used only by the browser and is not sent to the server.
2. **Parsing and caches.** The browser fills in missing parts, then checks its caches: the HSTS list, the HTTP cache and the DNS cache. A fresh cached copy means no request is made at all.
3. **DNS.** The hostname is turned into an IP address. The OS asks a recursive resolver, which asks the root, .com and authoritative servers if it has no cached answer. Each answer has a TTL. An A record gives an IPv4 address (32-bit, like `192.0.2.10`) and an AAAA record gives an IPv6 address (128-bit, like `2001:db8::10`).
4. **Connection.** For HTTP/1.1 and HTTP/2 the browser opens a TCP connection (SYN, SYN-ACK, ACK). HTTP/3 uses QUIC over UDP instead. A connection that is already open can be reused.
5. **TLS.** For HTTPS the server sends its certificate, the browser checks it, and both sides agree on keys so the traffic is encrypted. HTTP/1.1 sends one request at a time per connection, HTTP/2 sends many requests over one connection, and HTTP/3 does the same over QUIC.
6. **Request and response.** The browser sends a request and the server replies with a status line, headers and a body (the HTML). Simplified:

```
GET / HTTP/1.1
Host: www.google.com
Accept: text/html

HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
<!doctype html> ...
```

7. **GET and POST.** GET reads a resource and puts its data in the URL. POST sends data in the body, for example a form or an API call.
8. **Status codes.** 200 OK means success. 404 Not Found means nothing exists at that path. 500 Internal Server Error means the server failed.
9. **Rendering.** The HTML becomes the DOM and the CSS becomes the CSSOM. Each CSS, JavaScript, image or font file is another request that goes through the same steps. Layout and paint follow. CSS blocks rendering, and a plain script blocks HTML parsing unless it has `defer` or `async`.
10. **DevTools.** Press F12, open Network and reload. Clicking a request shows its headers, status code and Timing (DNS lookup, connection, SSL, waiting for the server, download).

Why it is useful. A 404 on `style.css` points to a wrong path, the Network tab shows what an API request sent and what came back, and the Disable cache option shows whether you are looking at an old copy.

Caching, connection reuse, HTTP versions and server behaviour can change the exact sequence, so not every request follows every step.

Section B: ARIA Roles and Web Accessibility

- **What it is.** Accessibility means a page can be used by people with different needs: blind or low-vision users, people who cannot use a mouse, deaf users, and people who find cluttered pages hard to follow. The standard is WCAG 2.2.
- **Semantic HTML.** Use elements for what they are. `header` is the top of the page, `nav` holds the main links, `main` holds the main content (one per page), `section` groups related content under a heading, and `footer` holds closing information. Screen readers can jump between these, and native elements like `button` and `a` already work with the keyboard.
- **ARIA.** It adds roles and states for assistive technology, such as `aria-label`, `aria-expanded` and `role="alert"`. It only changes what is announced and adds no behaviour. It helps when HTML has no element for the job: naming an icon-only button, showing whether a menu is open, announcing a form error, building tabs. It is unnecessary on native elements (`<button role="button">`), so use native HTML first.
- **Forms.** Every input gets a label linked with `for` and `id`. A placeholder is not a label.
- **Keyboard and focus.** Everything must work with Tab and Enter in a sensible order, and the focused item must be visible. Do not remove the outline without replacing it.
- **Alt text.** Informative images get a short description. Decorative images get `alt=""`.
- **Contrast.** At least 4.5:1 for normal text and 3:1 for large text. `#999` on white is 2.85:1 (fails). `#767676` is 4.54:1 (passes).

Example

```
<header>
  <h1>Your Name</h1>
  <nav aria-label="Main"> ...links... </nav>
</header>
<main id="main">
  <img src="profile.jpg" alt="Portrait of Your Name">
  <label for="email">Email</label>
  <input type="email" id="email" name="email" required>
</main>

a:focus-visible, button:focus-visible,
input:focus-visible { outline: 3px solid #1a56db; }
```

On a profile page this means a header with the name, a nav for the links, the About and Projects parts as sections inside main, a labelled contact form, and a footer.

Checklist

- [ ] One `h1`, headings in order, header, nav, main and footer used
- [ ] Native elements first, ARIA only when needed
- [ ] Every form field has a label
- [ ] Whole page works with the keyboard and focus is visible
- [ ] Images have the right alt text
- [ ] Text contrast is at least 4.5:1
- [ ] Page still works at 200% zoom

Section C: Responsive Web Design

- **What it is.** One page adapts to phone, tablet and desktop widths using flexible layouts, flexible images and media queries.
- **Selectors** choose which elements a rule styles: `p` (type), `.card` (class), `#about` (id), `nav a` (descendant), `a:hover` (state).
- **Box model.** Content, padding, border and margin. By default `width` covers only the content, so `width: 300px` with 20px padding on each side and a 1px border is 342px wide. `box-sizing: border-box` makes width include padding and border. Set it on everything.
- **Flexbox** lays items out in a row or column. `display: flex` makes the container, `justify-content` spaces items along the main axis, `align-items` aligns them on the cross axis, `gap` adds space between them and `flex-wrap` lets them move to the next line.
- **Flexible widths.** Use `max-width` with `margin-inline: auto` instead of fixed widths.
- **Viewport tag.** `<meta name="viewport" content="width=device-width, initial-scale=1">` goes in the head. Without it phones show a zoomed-out desktop layout.
- **Images.** `max-width: 100%; height: auto;` stops them overflowing.
- **Media queries** apply CSS at certain widths. Mobile-first means the base CSS is for phones and `@media (min-width: 48em)` adds changes for larger screens. Pick breakpoints where the layout breaks, not by device model.
- **Common mistakes.** Missing viewport tag, fixed pixel widths, no border-box, images overflowing, tiny tap targets, disabling zoom.
- **Testing.** In DevTools press F12, then Ctrl+Shift+M for the device toolbar to try different widths. Check on a real phone too.

Example for the profile page

```
*, *::before, *::after { box-sizing: border-box; }
img { max-width: 100%; height: auto; }
main { max-width: 60rem; margin-inline: auto; padding: 0 1rem; }
.nav-list { display: flex; flex-wrap: wrap; gap: 1rem; }
.cards { display: flex; flex-wrap: wrap; gap: 1rem; }
.card { flex: 1 1 16rem; border: 1px solid #ccc; padding: 1rem; }
.about { display: flex; flex-direction: column; gap: 1rem; }
@media (min-width: 48em) { .about { flex-direction: row; } }
```

The nav links wrap instead of running off the screen, the cards sit one per row on a phone and several per row on a desktop, and the About photo and text stack on phones and sit side by side on wide screens.

Testing checklist

- **Mobile (about 360px):** no sideways scroll, text readable, links easy to tap
- **Tablet (about 768px):** layout changes cleanly at the breakpoint
- **Desktop (1280px and wider):** content centred, lines not too long
- **All:** zooming to 200% still works

Conclusion

These notes follow a page from request to screen. Section A explains how the page reaches the browser and how to inspect it in DevTools. Section B covers making it usable for everyone with semantic HTML, labels, visible focus and enough contrast. Section C covers making the layout fit any screen with border-box, flexbox, the viewport tag and media queries. All three ideas apply directly to the profile page.

References

MDN, An overview of HTTP: https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview; MDN, HTTP response status codes: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status; RFC 9110, HTTP Semantics: https://www.rfc-editor.org/rfc/rfc9110; RFC 8446, TLS 1.3: https://www.rfc-editor.org/rfc/rfc8446; web.dev, Understanding the critical rendering path: https://web.dev/learn/performance/understanding-the-critical-path; Chrome DevTools, Network reference: https://developer.chrome.com/docs/devtools/network/reference; W3C, WCAG 2.2: https://www.w3.org/TR/WCAG22/; W3C WAI, ARIA Authoring Practices, Read Me First: https://www.w3.org/WAI/ARIA/apg/practices/read-me-first/; MDN, Responsive web design: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design; MDN, Introduction to the CSS box model: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_box_model/Introduction_to_the_CSS_box_model.